

Deployment Guide

This is a Laravel app that pulls data from the **Osool-B2G DWH sync API**, transforms it into a Postgres star schema, and serves dashboards backed by **OpenSearch**. It runs as a Docker Compose stack: PHP-FPM (`app`) + Nginx + PostgreSQL 16 + OpenSearch 2.18.

1. Prerequisites

Install on the host before anything else:

Requirement	Notes
Docker Engine 24+ and Docker Compose v2	The whole stack runs in containers; no PHP/Node needed on the host.
Git	To clone the repo.
<code>vm.max_map_count = 262144</code>	Required by OpenSearch or its container will fail to start (see below).
RAM: 4 GB minimum (8 GB recommended)	OpenSearch reserves 512 MB heap; the sync of <code>work_orders</code> (~90k rows) can use up to 2 GB PHP memory.
Outbound network access to the Osool API	The host must be able to reach <code>OSOOL_BASE_URL</code> (the DWH sync endpoint) over HTTPS.

Recommended server specifications

All four services run on a single host. Sizing is driven by OpenSearch (512 MB heap by default, plus Lucene off-heap), PostgreSQL, and the data sync — a full `work_orders` backfill (~90k rows × 100+ columns) can use up to **2 GB** of PHP memory, and OpenSearch reindexing builds a new index alongside the old one before swapping (transient ~2× index size on disk).

Resource	Minimum (PoC / staging)	Recommended (production)	Large / growth
CPU	2 vCPU	4 vCPU	8 vCPU
RAM	4 GB	8 GB	16 GB
Disk	20 GB SSD	50 GB SSD	100 GB+ SSD (NVMe)
OS	Linux x86-64 (Ubuntu 22.04 LTS / Debian 12)	same	same

Notes:

- SSD is required**, not optional — both OpenSearch and PostgreSQL are random-I/O heavy.
- 4 GB is the floor** and is tight: a sync running while OpenSearch reindexes can exhaust it. Use 8 GB for any real workload.
- For heavier data volumes, raise the OpenSearch heap (`OPENSEARCH_JAVA_OPTS=-Xms2g -Xmx2g` in `docker-compose.yml`) and keep heap ≤ 50% of host RAM.
- Disk grows with three copies of the data: `raw` JSONB landing + `marts` star schema + OpenSearch indices. Leave headroom for the transient second index during reindex.
- Single-host Docker Compose is assumed. For HA or large scale, split PostgreSQL and OpenSearch onto dedicated nodes (out of scope for this guide).

Set the OpenSearch kernel parameter (persists across reboots):

```
sudo sysctl -w vm.max_map_count=262144
echo 'vm.max_map_count=262144' | sudo tee /etc/sysctl.d/99-opensearch.conf
```

Ports

Port	Service	Exposure
8080	Nginx → the app UI	Public — this is how users reach the dashboard (<code>http://<server>:8080</code>).
5432	PostgreSQL	Published to the host by <code>docker-compose.yml</code> . Firewall this off from the public internet in production.
9200	OpenSearch	Published to the host by <code>docker-compose.yml</code> . Firewall this off as well.

Security plugin is disabled on OpenSearch (`plugins.security.disabled=true`) for simplicity, so port 9200 has no authentication. Do not expose it publicly.

2. Configuration (`.env`)

```
git clone <repo-url>
cd opensearch2
cp .env.example .env
```

Then edit `.env` . The values below are the ones that matter for a server deployment — the rest of the Laravel defaults are fine.

```

# --- Application ---
APP_NAME="Osool Dashboard"
APP_ENV=production
APP_DEBUG=false
APP_KEY=                # filled in by `php artisan key:generate` (step 3)
APP_URL=http://<server-host-or-ip>:8080

# --- Database (PostgreSQL) ---
# IMPORTANT: .env.example ships with sqlite. This MUST be pgsql or the app silently
# uses SQLite and every query fails. docker-compose injects the host/port/credentials
# below into the container, but it does NOT set DB_CONNECTION - so set it here.
DB_CONNECTION=pgsql
DB_HOST=postgres
DB_PORT=5432
DB_DATABASE=opensearch2
DB_USERNAME=postgres
DB_PASSWORD=secret      # matches POSTGRES_PASSWORD in docker-compose.yml

# --- OpenSearch ---
OPENSEARCH_SCHEME=http
OPENSEARCH_HOST=opensearch
OPENSEARCH_PORT=9200
# OPENSEARCH_USER=      # only if you enable the security plugin
# OPENSEARCH_PASSWORD=
OPENSEARCH_INDEX_PREFIX=osoool_ # index/alias prefix; keep stable across deploys

# --- Osool DWH Sync API (REQUIRED to load real data) ---
# Where `sync:run` / `sync:cycle` pull from, with HMAC auth.
OSOOL_BASE_URL=https://<osoool-dwh-host>
OSOOL_HMAC_SECRET=<shared-secret-from-osoool> # must match the DWH side exactly
# Optional tuning (defaults shown):
# OSOOL_SYNC_PAGE_SIZE=500
# OSOOL_SYNC_TIMEOUT=30
# OSOOL_SYNC_RETRIES=3
# OSOOL_CURSOR_OVERLAP=600

# --- SSO (Osool ↔ Dashboard login) - SEPARATE from the sync secret above ---
SSO_HMAC_SECRET=<shared-sso-secret> # must match Osool's SSO_HMAC_SECRET
OSOOL_URL=https://test.osoool.cloud # Osool base URL for SSO redirects
# OSOOL_INTERNAL_URL=              # defaults to OSOOL_URL; set if back-channel uses a private IP

# --- Background processing (defaults are correct) ---
QUEUE_CONNECTION=database
SESSION_DRIVER=database
CACHE_STORE=database

```

Two different HMAC secrets. `OSOOL_HMAC_SECRET` authenticates the **data sync** (`config/osoool.php`); `SSO_HMAC_SECRET` authenticates **user login / back-channel logout** (`config/sso.php`). They are not interchangeable — get both from the Osool team.

Generate a secret with: `php -r "echo bin2hex(random_bytes(32));"`

3. Build & Initialize

```
# Build images and start all containers
docker-compose up -d --build

# Generate the application key (writes APP_KEY into .env)
docker exec opensearch2-app php artisan key:generate

# Create all schemas, extensions, tables, and materialized views
docker exec opensearch2-app php artisan migrate

# Seed the calendar dimension table (required before loading data)
docker exec opensearch2-app php artisan dwh:seed-calendar
```

`migrate` creates the `raw`, `marts`, `reports`, `dwh`, `auth`, `pii` schemas and the `citext` extension automatically — no manual SQL needed.

Verify the stack is healthy before loading data:

```
docker-compose ps # all containers "healthy"/"running"
docker exec opensearch2-app curl -s opensearch:9200/_cluster/health # expect "status":"green"/"yellow"
```

4. Load Real Data from the API

Option A — Full cycle in one command (recommended)

Runs every resource in dependency-stage order, then reindexes each affected OpenSearch index once at the end:

```
docker exec opensearch2-app php artisan sync:cycle
```

Option B — Resource by resource (if you need to run selectively)

Run in this order — each command pulls from the API, transforms into the marts schema, and reindexes the dependent OpenSearch indices automatically:

```

# Stage 1 - Regions
docker exec opensearch2-app php artisan sync:run regions

# Stage 2 - Reference / lookup tables
docker exec opensearch2-app php artisan sync:run cities
docker exec opensearch2-app php artisan sync:run service-providers
docker exec opensearch2-app php artisan sync:run users
docker exec opensearch2-app php artisan sync:run projects-details
docker exec opensearch2-app php artisan sync:run asset-statuses
docker exec opensearch2-app php artisan sync:run contract-types

# Stage 3 - User-project links
docker exec opensearch2-app php artisan sync:run user-projects

# Stage 4 - Properties
docker exec opensearch2-app php artisan sync:run properties

# Stage 5 - Property children & asset metadata
docker exec opensearch2-app php artisan sync:run property-buildings
docker exec opensearch2-app php artisan sync:run asset-categories
docker exec opensearch2-app php artisan sync:run asset-names
docker exec opensearch2-app php artisan sync:run priorities

# Stage 6 - Transactional data (heaviest)
docker exec opensearch2-app php artisan sync:run work-orders
docker exec opensearch2-app php artisan sync:run assets
docker exec opensearch2-app php artisan sync:run lease-contract-details # raw-only feeder; must run before
docker exec opensearch2-app php artisan sync:run commercial-contracts
docker exec opensearch2-app php artisan sync:run contracts

# Stage 7 - Contract children
docker exec opensearch2-app php artisan sync:run payment-details
docker exec opensearch2-app php artisan sync:run contract-months
docker exec opensearch2-app php artisan sync:run packages

```

5. OpenSearch Reindex (standalone)

Only needed if the OpenSearch index gets out of sync. `sync:run` and `sync:cycle` handle reindexing automatically — skip this if you used them.

```

docker exec opensearch2-app php artisan os:reindex work_orders
docker exec opensearch2-app php artisan os:reindex properties
docker exec opensearch2-app php artisan os:reindex assets
docker exec opensearch2-app php artisan os:reindex users
docker exec opensearch2-app php artisan os:reindex commercial_contracts
docker exec opensearch2-app php artisan os:reindex installments
docker exec opensearch2-app php artisan os:reindex contracts
docker exec opensearch2-app php artisan os:reindex projects

```

6. Queue Worker

The queue connection is `database`. Start the worker so background jobs are processed:

```
docker exec opensearch2-app php artisan queue:work --sleep=3 --tries=3 --timeout=90
```

To run it in the background (detached):

```
docker exec -d opensearch2-app php artisan queue:work --sleep=3 --tries=3 --timeout=90
```

Check for / retry failed jobs:

```
docker exec opensearch2-app php artisan queue:failed
docker exec opensearch2-app php artisan queue:retry all
```

7. Scheduler

The scheduler runs `sync:cycle` automatically every 30 minutes (no overlap). Start it with:

```
docker exec -d opensearch2-app php artisan schedule:work
```

Alternatively, add a system cron entry that calls `schedule:run` every minute:

```
* * * * * docker exec opensearch2-app php artisan schedule:run >> /dev/null 2>&1
```

Production note: `docker exec -d` workers die if the `app` container restarts. For a durable setup, run the worker and scheduler as their own long-lived services (e.g. extra `docker-compose` services using the same image with `command: php artisan queue:work` / `schedule:work`, with `restart: unless-stopped`) or under a process manager like supervisord.

8. Maintenance

```
# Clear application cache
docker exec opensearch2-app php artisan cache:clear

# View sync cycle log
docker exec opensearch2-app tail -f storage/logs/sync-cycle.log

# Wipe ALL DWH data (Postgres + OpenSearch) – use only to start fresh
docker exec opensearch2-app php artisan dwh:wipe --force

# Container status / logs
docker-compose ps
docker-compose logs -f

# Stop containers
docker-compose down

# Stop and remove volumes (wipes database + OpenSearch data)
docker-compose down -v
```

Quick Reference

Goal	Command
Set kernel param for OpenSearch	<code>sudo sysctl -w vm.max_map_count=262144</code>
Start everything	<code>docker-compose up -d --build</code>
Initialize DB	<code>php artisan migrate && php artisan dwh:seed-calendar</code>
Full data sync	<code>php artisan sync:cycle</code>
Reindex OpenSearch only	<code>php artisan os:reindex <entity></code>
Start queue worker	<code>php artisan queue:work</code>
Start scheduler	<code>php artisan schedule:work</code>
Wipe and start fresh	<code>php artisan dwh:wipe --force</code>